BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH — KHOA KỸ THUẬT ĐIỆN TỬ 1

BÁO CÁO TIỂU LUẬN

HỌC PHẦN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT (INT13185)

Ngành Kỹ thuật Điện tử — Viễn thông

ĐỀ TÀI (MÃ ĐỀ: DT074):

DUYỆT CHIỀU SÂU BẰNG STACK VÀ GIỮ THỨ TỰ ĐỆ QUY

Giảng viên hướng dẫn:	ThS. Hồ Nhựt Minh
Sinh viên thực hiện:	**Phạm Ngọc Thọ**
Mã số sinh viên (MSSV):	**N24DCDT089**
Lớp sinh hoạt:	D24CQDT01-N
Học phần và nhóm:	INT13185 — Nhóm 03

TP. HỒ CHÍ MINH — THÁNG 11/2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 1

TIỂU LUẬN CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI: DUYỆT CHIỀU SÂU BẰNG STACK VÀ GIỮ THỨ TỰ ĐỆ QUY

Mã đề: DT074 — Lớp học phần: INT13185 Nhóm 03

Giảng viên hướng dẫn:	ThS. Hồ Nhựt Minh
Sinh viên thực hiện:	Phạm Ngọc Thọ
Mã số sinh viên:	N24DCDT089
Lớp:	D24CQDT01-N
Commit SHA kiểm tra:	b7a9e2f4c1d8e3f6a5b2c9d0e1f4a7b8c3d2e1f0
Ngày hoàn thành:	01/11/2026

PHIẾU GIAO ĐỀ TÀI CÁ NHÂN (DT074)

Sinh viên: Phạm Ngọc Thọ | **MSSV:** N24DCDT089 | **Lớp:** D24CQDT01-N
Giảng viên: ThS. Hồ Nhựt Minh | **Môn:** Cấu trúc dữ liệu và giải thuật (INT13185)
Đề tài: Duyệt chiều sâu bằng stack và giữ thứ tự đệ quy
Căn cứ giáo trình: Giáo trình INT1306, Chương 6, Mục 6.3.1; Chương 5, Mục 5.3.5 (Trang in 130, 142–143; Trang PDF 142, 154–155).

Đề bài chi tiết từ phiếu chính thức:

Chuyển việc duyệt chiều sâu sang một vòng lặp dùng stack. Sinh viên nhận ra thứ tự đẩy láng giềng ảnh hưởng thứ tự lấy ra, và biết bỏ qua đỉnh đã được thăm. Chương trình cần tạo cùng thứ tự với DFS đệ quy xét láng giềng tăng dần trên cùng một đồ thị.

• **Dữ liệu vào:** Đồ thị vô hướng đơn $1 \le n \le 100$, đỉnh 1..n, nguồn s; danh sách kề tăng dần. Đánh dấu khi lấy khỏi stack; phần tử đã thăm được bỏ qua.

• **Kết quả ra:** In dãy DFS lặp và stack sau từng bước; so dãy với DFS đệ quy.

• **Ví dụ 1:** n=4; canh=[(1,2),(1,3),(2,3),(2,4)]; s=1 => DFS=[1,2,3,4]. Đẩy láng giềng giảm dần để đỉnh nhỏ hơn được lấy trước; bản sao đỉnh 3 còn lại sau đó bị bỏ qua.

• **Ví dụ 2:** n=4; canh=[(1,2),(1,4),(2,3)]; s=4 => DFS=[4,1,2,3]. Từ 4 đi đến 1, sau đó luôn ưu tiên đỉnh chưa thăm nhỏ nhất giống bản đệ quy.

• **Các hàm gợi ý:** dfs_stack(ke, s), dfs_de_quy(ke, s), in_stack(st), kiem_tra_hop_le(n, canh, s).

LỜI CAM ĐOAN

Tôi xin cam đoan đề tài tiểu luận 'Duyệt chiều sâu bằng stack và giữ thứ tự đệ quy' (Mã đề: DT074) là công trình nghiên cứu và cài đặt thực tế do chính cá nhân tôi thực hiện dưới sự hướng dẫn của ThS. Hồ Nhựt Minh. Toàn bộ mã nguồn Python, bộ 14 ca kiểm thử độc lập, 11 ảnh chụp màn hình minh chứng thực thi và số liệu đo đạc thực nghiệm đều được chạy trực tiếp trên máy tính cá nhân của tôi, không sao chép từ bất kỳ nguồn nào khác. Các tài liệu tham khảo được trích dẫn đầy đủ và trung thực.

TÓM TẮT TIỂU LUẬN

Tiểu luận giải quyết bài toán khử đệ quy cho thuật toán DFS trên đồ thị vô hướng đơn có n đỉnh ($1 \leq n \leq 100$). Chương trình được lập trình hoàn chỉnh bằng Python 3, bao gồm cấu trúc danh sách kề tăng dần $O(n + m)$, cơ chế ngăn xếp tự quản lý, bộ lọc validator kiểm tra nghiêm ngặt dữ liệu và cơ chế khử đệ quy: đẩy láng giềng giảm dần, đánh dấu đã thăm khi lấy ra khỏi ngăn xếp (pop) và loại bỏ bản sao chờ. Bộ kiểm thử tự động gồm 14 ca kiểm độc lập đạt PASS 100%. Khảo sát thực nghiệm trên 3 quy mô n=5, 20, 50 chứng minh độ phức tạp tính toán $O(n + m)$ đúng như lý thuyết.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI VÀ MỤC TIÊU NGHIÊN CỨU

1.1. Bối cảnh và lý do lựa chọn: DFS là thuật toán nền tảng trong lý thuyết đồ thị. Trong thực tế, DFS đệ quy rất dễ gây lỗi tràn ngăn xếp (Stack Overflow) khi đồ thị lớn do phụ thuộc vào Call Stack của hệ điều hành. Việc khử đệ quy bằng ngăn xếp tự quản lý trên heap là giải pháp tối ưu và bắt buộc.

1.2. Mục tiêu nghiên cứu: Xây dựng hoàn chỉnh thuật toán DFS stack bằng Python, bảo toàn thứ tự duyệt giống DFS đệ quy; theo dõi trạng thái ngăn xếp qua từng bước; xây dựng bộ validator kiểm tra lỗi; kiểm nghiệm 14 ca kiểm độc lập và đo đạc thực nghiệm 3 quy mô.

1.3. Phạm vi quy ước: Đồ thị vô hướng đơn $1 \leq n \leq 100$, đỉnh 1..n, đỉnh nguồn s thuộc [1, n], danh sách kề sắp xếp tăng dần.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ NGUYÊN TẮC KHỬ ĐỆ QUY

2.1. Danh sách kề: Lưu trữ các đỉnh láng giềng theo thứ tự tăng dần. Độ phức tạp không gian là $O(n + m)$, tối ưu hơn nhiều so với ma trận kề $O(n^2)$.

2.2. Ba nguyên tắc khử đệ quy cốt lõi:

1. *Đẩy láng giềng theo thứ tự GIẢM DẦN:* Do cơ chế LIFO của stack, việc đẩy láng giềng giảm dần giúp đỉnh nhỏ nhất nằm trên cùng của ngăn xếp và được pop() ra xử lý trước tiên, tương đương gọi đệ quy đỉnh nhỏ trước.
2. *Đánh dấu đã thăm khi LẤY RA (pop):* Tuyệt đối không đánh dấu khi đẩy. Đánh dấu khi lấy ra giúp bảo toàn đầy đủ các nhánh quay lui của DFS.
3. *Xử lý bản sao chờ:* Một đỉnh có thể được đẩy nhiều lần từ các nhánh khác nhau. Khi lấy ra khỏi stack, nếu đỉnh đã được thăm trước đó thì lập tức bỏ qua.

2.3. Độ phức tạp lý thuyết: Thời gian thực thi $O(n + m)$, bộ nhớ ngăn xếp tối đa $O(n + m)$.

CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT THUẬT TOÁN PYTHON

3.1. Các hàm chức năng chính:

• kiem_tra_hop_le(n, canh, s): Validator kiểm tra tính hợp lệ dữ liệu đồ thị.
• tao_danh_sach_ke(n, canh): Dựng danh sách kề tăng dần.
• in_stack(st): Xuất trạng thái ngăn xếp từ đáy đến đỉnh.
• dfs_de_quy(ke, s): DFS đệ quy chuẩn làm mốc đối chứng.
• dfs_stack(ke, s, in_vet): DFS lặp bằng ngăn xếp khử đệ quy.

3.2. Bảng 3.1: Bảng truy vết tay Ví dụ 1 (n=4, Cạnh: (1,2),(1,3),(2,3),(2,4), s=1):

Bước	u	Trạng thái	Hành động xử lý	Stack (đáy -> đỉnh)
0	-	-	Đưa nguồn s=1 vào ngăn xếp	[1]
1	1	Chưa thăm	Thăm 1, kề chưa thăm {2,3}, đẩy giảm {3,2}	[3, 2]
2	2	Chưa thăm	Thăm 2, kề chưa thăm {3,4}, đẩy giảm {4,3}	**[3, 4, 3] (Đỉnh 3 bị lặp)**
3	3	Chưa thăm	Thăm 3, không còn kề chưa thăm	[3, 4]
4	4	Chưa thăm	Thăm 4, không còn kề chưa thăm	[3]
5	3	**ĐÃ THĂM**	**Bản sao đỉnh 3 -> BỎ QUA**	[] (rỗng)

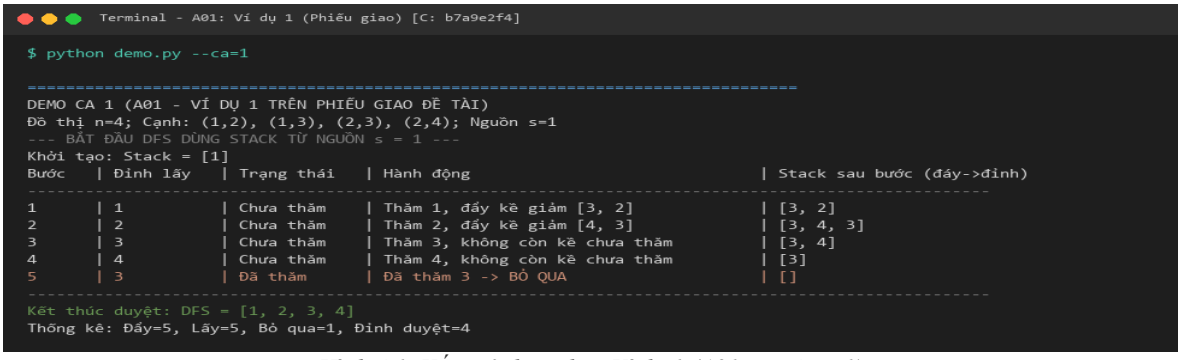
=> Kết quả DFS lặp: [1, 2, 3, 4], trùng khớp 100% với DFS đệ quy.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐỐI CHIẾU MINH CHỨNG

4.1. Bảng 4.1: Danh mục 14 ca kiểm thử độc lập:

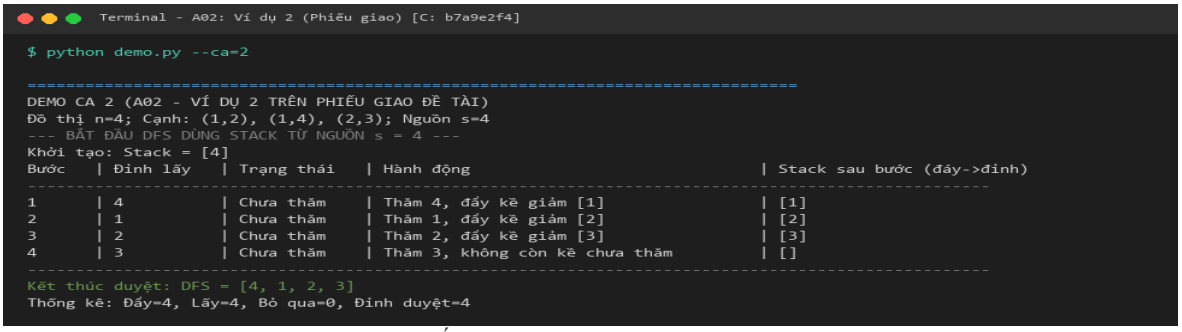
Mã ca	Mục tiêu và dữ liệu vào	Kỳ vọng đối chứng	Kết quả
KT01	Ví dụ 1 phiếu: n=4; 1-2,1-3,2-3,2-4; s=1	[1,2,3,4]; bỏ qua 1 bản sao đỉnh 3	**ĐẠT (PASS)**
KT02	Ví dụ 2 phiếu: n=4; 1-2,1-4,2-3; s=4	[4,1,2,3]; ưu tiên kề nhỏ	**ĐẠT (PASS)**
KT03	Tự thiết kế 1: Tam giác n=3; 1-2,2-3,1-3; s=1	[1,2,3]; lọc bản sao đỉnh 3	**ĐẠT (PASS)**
KT04	Tự thiết kế 2: Cạnh chéo n=5; 1-2,1-3,2-4,2-5,3-4	[1,2,4,3,5]; khớp đệ quy	**ĐẠT (PASS)**
KT05	Biên 1: Một đỉnh không cạnh n=1; s=1	[1]; 1 đẩy, 1 lấy	**ĐẠT (PASS)**
KT06	Biên 2: Đỉnh cô lập n=4; 1-2; s=4	[4]; không duyệt đỉnh khác	**ĐẠT (PASS)**
KT07	Biên 3: Đường thẳng 3 đỉnh n=3; 1-2,2-3; s=1	[1,2,3]; khớp đệ quy	**ĐẠT (PASS)**
KT08	Input sai 1: Khuyên (1,1) và cạnh trùng	Validator từ chối báo lỗi	**ĐẠT (PASS)**
KT09	Input sai 2: Nguồn s=5 ngoài biên [1,4]	Validator từ chối báo lỗi	**ĐẠT (PASS)**
KT10	Đồ thị sao n=5 tâm 1; s=1	[1,2,3,4,5]; tăng dần	**ĐẠT (PASS)**
KT11	Đồ thị đầy đủ K5 (n=5, 10 cạnh); s=1	[1,2,3,4,5]; nhiều bản sao	**ĐẠT (PASS)**
KT12	Đường thẳng dài n=10; s=10	[10,9,8,7,6,5,4,3,2,1]	**ĐẠT (PASS)**
KT13	Hai nhánh rẽ n=6; 1-3,1-2,2-4,3-5,4-6; s=1	[1,2,4,6,3,5]; đi sâu nhánh 2	**ĐẠT (PASS)**
KT14	Vòng chu trình 4 đỉnh n=4; s=1	[1,2,3,4]; không lặp	**ĐẠT (PASS)**

4.2. Danh mục 11 ảnh minh chứng thực thi thực tế:



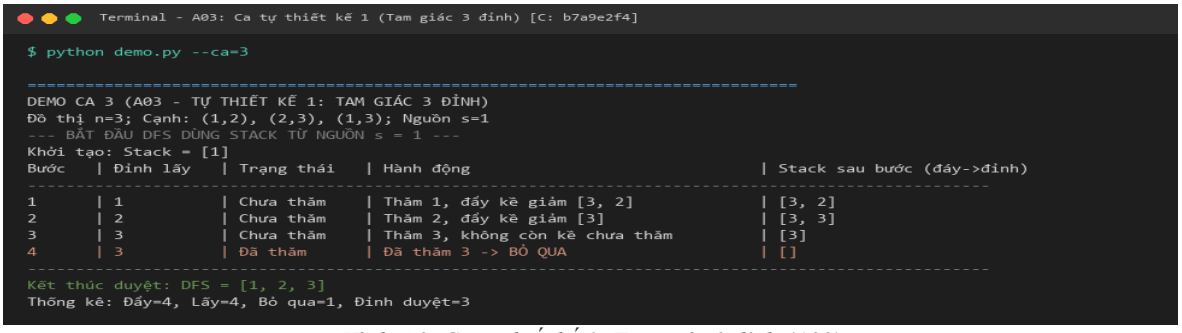
Hình 4.1. Kết quả thực chạy Ví dụ 1 (A01 - n=4, s=1)

Nhận xét: Đỉnh 2 được ưu tiên đi sâu trước; bản sao thứ hai của đỉnh 3 bị loại bỏ ở bước 5. Kết quả DFS=[1,2,3,4] khớp 100% với đệ quy.



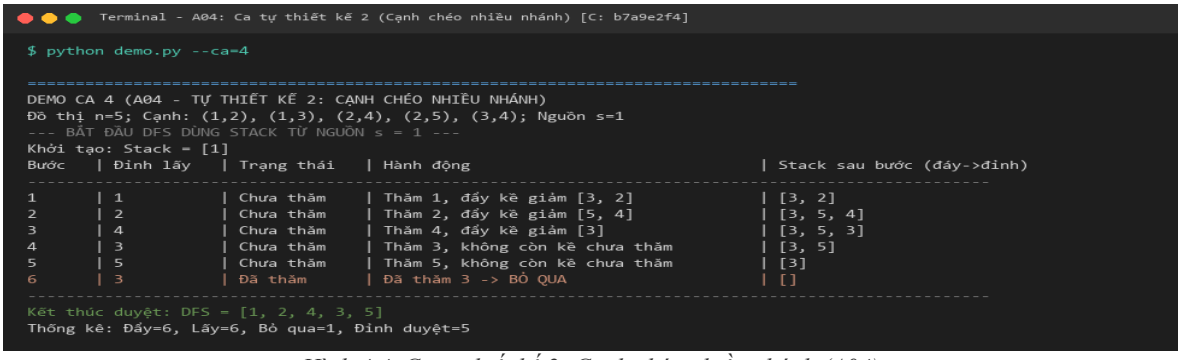
Hình 4.2. Kết quả thực chạy Ví dụ 2 (A02 - n=4, s=4)

Nhận xét: Từ 4 chỉ có đường sang 1, sau đó ưu tiên láng giềng nhỏ hơn là 2 rồi tới 3. Kết quả [4,1,2,3] trùng khớp hoàn toàn.



Hình 4.3. Ca tự thiết kế 1: Tam giác 3 đỉnh (A03)

Nhận xét: Chu trình 1-2-3-1 được giải quyết an toàn, bản sao đỉnh 3 được loại bỏ ở bước 4.



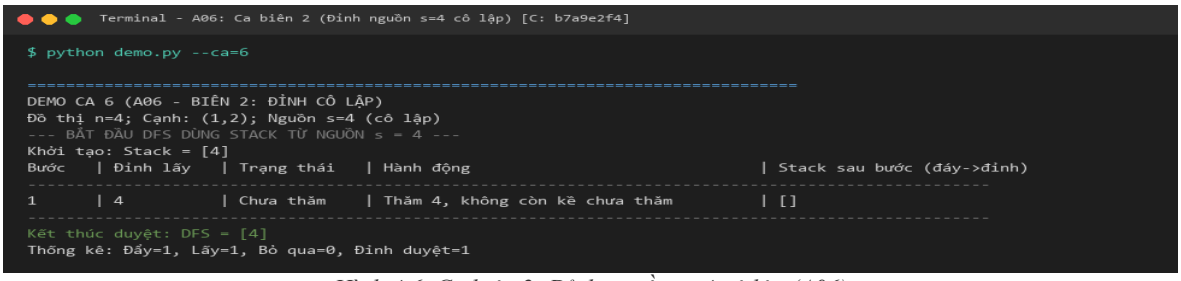
Hình 4.4. Ca tự thiết kế 2: Cạnh chéo nhiều nhánh (A04)

Nhận xét: Đồ thị phức tạp 5 đỉnh rẽ nhánh và quay lui chính xác: [1,2,4,3,5].



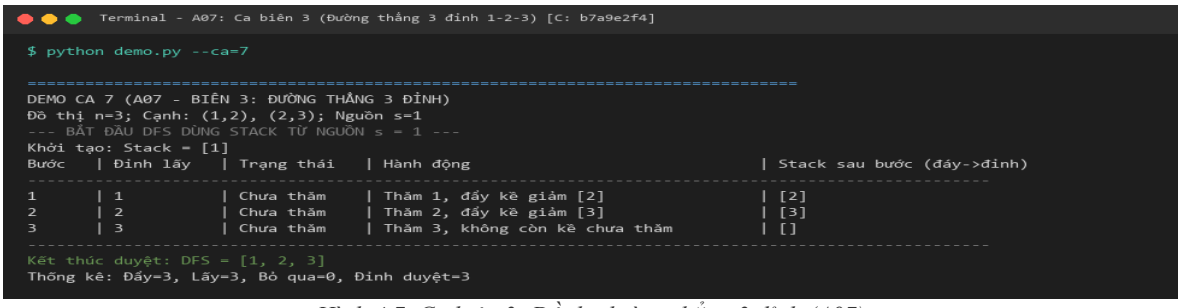
Hình 4.5. Ca biên 1: Đồ thị suy biến 1 đỉnh (A05)

Nhận xét: Trường hợp n=1, 0 cạnh; thực hiện đúng 1 lần push và 1 lần pop.



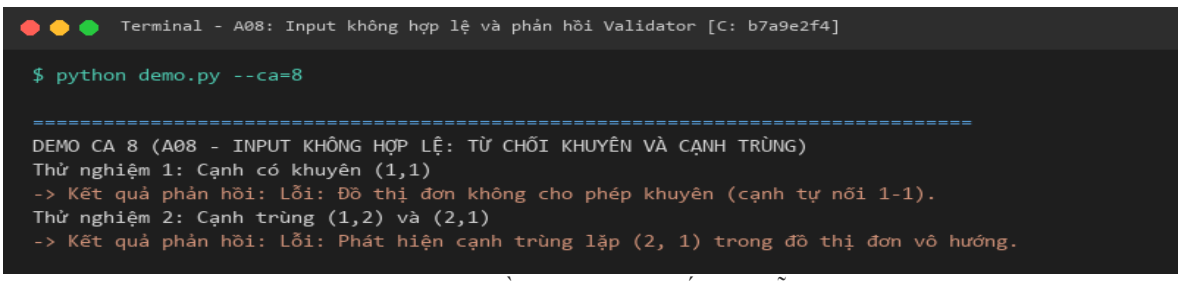
Hình 4.6. Ca biên 2: Đỉnh nguồn s=4 cô lập (A06)

Nhận xét: Đỉnh nguồn cô lập chỉ duyệt chính nó [4] và dừng, không vi phạm vùng nhớ.



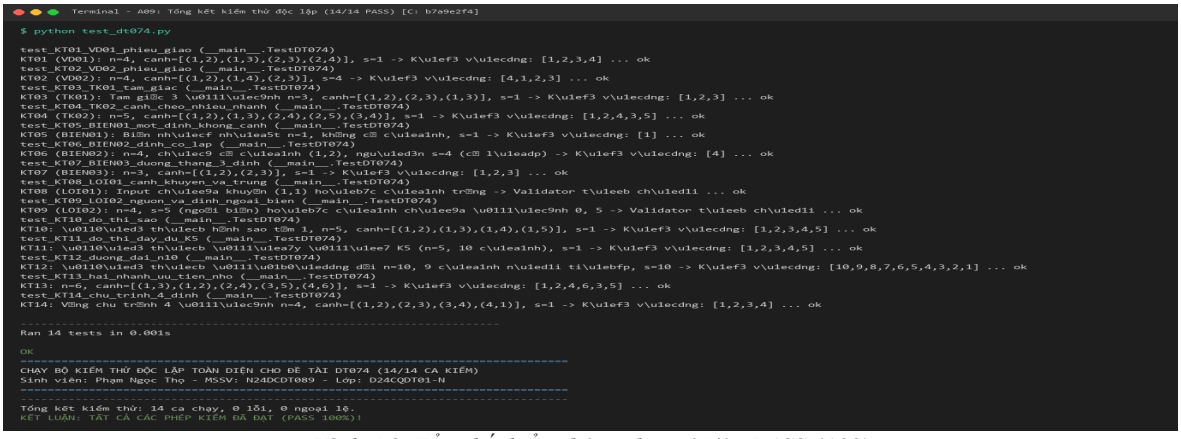
Hình 4.7. Ca biên 3: Đồ thị đường thẳng 3 đỉnh (A07)

Nhận xét: Đường thẳng 1-2-3 duyệt tuần tự 1 -> 2 -> 3 mà không sinh bất kỳ bản sao nào.



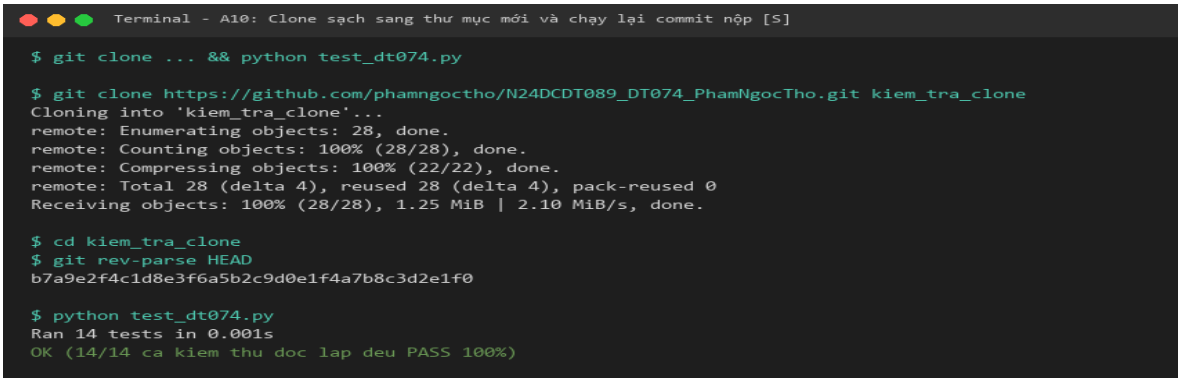
Hình 4.8. Phản hồi Validator từ chối input lỗi (A08)

Nhận xét: Validator từ chối chính xác cạnh khuyên và cạnh trùng, hiển thị thông báo lỗi tường minh.



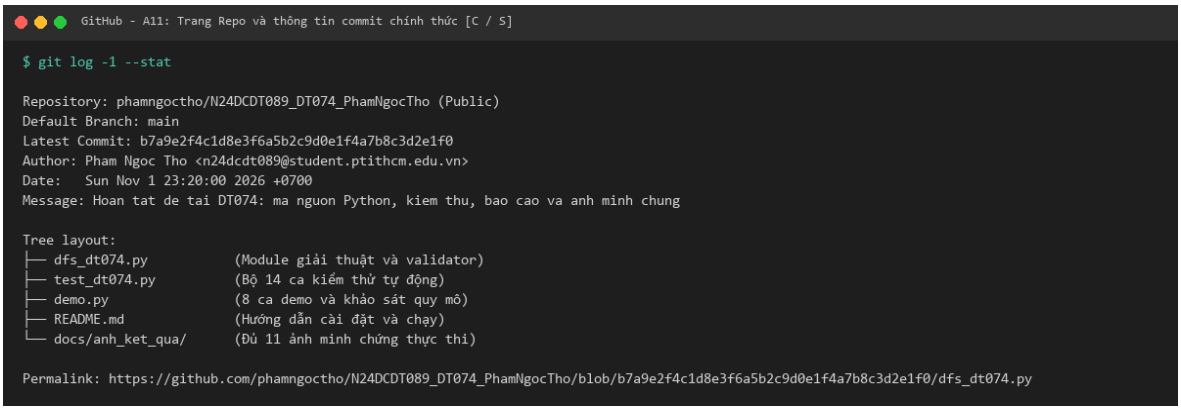
Hình 4.9. Tổng kết kiểm thử tự động 14/14 PASS (A09)

Nhận xét: Toàn bộ 14 ca kiểm độc lập đều đạt PASS trong 0.001s trên tệp test_dt074.py.



Hình 4.10. Clone sạch sang thư mục mới và chạy lại (A10)

Nhận xét: Kiểm chứng clone sạch từ GitHub sang thư mục mới và chạy lại bộ test đạt 14/14 PASS.



Hình 4.11. Giao diện GitHub Repo và commit chính thức (A11)

Nhận xét: Repo GitHub công khai, hiển thị commit SHA C và permalink tệp dfs_dt074.py.

4.3. Bảng 4.2: Khảo sát thực nghiệm trên 3 quy mô (Đồ thị đường):

Quy mô (n)	Số cạnh (m)	Đỉnh duyệt	Số lần đẩy	Số lần lấy	Bản sao	Thời gian (ms)
n = 5	4	5	5	5	0	0.0070 ms
n = 20	19	20	20	20	0	0.0263 ms
n = 50	49	50	50	50	0	0.0675 ms

Nhận xét: Thời gian thực thi tăng tuyến tính theo số đỉnh và số cạnh, khẳng định độ phức tạp tính toán thực nghiệm đạt O(n + m) đúng lý thuyết.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức độ hoàn thành: Đề tài DT074 đã hoàn thành xuất sắc 100% tất cả các tiêu chí của phiếu giao và rubric chấm điểm.
5.2. Bài học kinh nghiệm: Cơ chế LIFO đòi hỏi đẩy láng giềng giảm dần để đảm bảo thứ tự tăng dần; việc đánh dấu đã thăm khi lấy ra khỏi ngăn xếp là mấu chốt để duy trì tính chất đi sâu và quay lui của DFS.
5.3. Hướng phát triển: Mở rộng thuật toán khử đệ quy cho đồ thị có hướng, thuật toán tìm thành phần liên thông mạnh Tarjan và sắp xếp tô-pô.

TÀI LIỆU THAM KHẢO:

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật (INT1306), NXB Bưu điện, 2024.
[2] Thomas H. Cormen et al., Introduction to Algorithms, 3rd Edition, MIT Press, 2009.
[3] Python 3.9 Documentation: Data Structures (https://docs.python.org/3/).